

SwimPay SDK developer integration guide

This guide explains how a developer should integrate SwimPay into:

- ? a web/backend application;
- ? an external Android merchant app.

SwimPay is a Payment Signal Engine. It is not a bank, PSP, wallet, payment initiator, SBP integration or official bank confirmation system.

V1 is manual-confirmation-only:

- ? the buyer checkout can create and arm a payment session;
- ? the Android Receiver can capture, redact, sign and upload allowed bank notification signals;
- ? the backend can create a merchant review when a signal matches an active payment intent;
- ? the merchant must confirm manually;
- ? `payment.confirmed` is emitted only after merchant manual confirmation.

Every public payment webhook includes:

```
{
  "confirmation_type": "notification_signal",
  "official_bank_confirmation": false
}
```

1. What the Developer Integration screen provides

The SwimPay merchant app / Developer Integration Wizard provides the values needed by an external developer.

Required values:

```
SWIMPAY_STAGING_API_BASE_URL=https://staging.swimpay.pro
SWIMPAY_STAGING_SECRET_KEY=sk...
SWIMPAY_STAGING_WEBHOOK_SECRET=whsec...
SWIMPAY_WEBHOOK_URL=https://your-backend.example/webhooks/swimpay
EXTERNAL_APP_BASE_URL=https://your-backend.example
SWIMPAY_PUBLIC_WEBHOOK_EVENTS=payment.confirmed,payment.rejected,payment.expired
```

Meaning:

Value	Owner	Where it is used
-------	-------	------------------

---	---	---
-----	-----	-----

`SWIMPAY_STAGING_API_BASE_URL`	SwimPay	Merchant backend calls SwimPay API
--------------------------------	---------	------------------------------------

`SWIMPAY_STAGING_SECRET_KEY`	SwimPay	Merchant backend creates orders
------------------------------	---------	---------------------------------

`SWIMPAY_STAGING_WEBHOOK_SECRET`	SwimPay	Merchant backend verifies webhooks
----------------------------------	---------	------------------------------------

| `SWIMPAY\_WEBHOOK\_URL` | Merchant developer | Public HTTPS endpoint that receives SwimPay webhooks |

| `EXTERNAL\_APP\_BASE\_URL` | Merchant developer | Your own app/backend base URL |

| `SWIMPAY\_PUBLIC\_WEBHOOK\_EVENTS` | SwimPay | Public final events supported in V1 |

Important:

? The full API key and full webhook secret are show-once values.

? If the developer only has masked values such as `sk\_\*\*1234` or `whsec\_\*\*1234`, they cannot integrate. Rotate/create the value again and copy it immediately.

? Secrets must stay on the merchant backend.

? Never put `SWIMPAY\_STAGING\_SECRET\_KEY` or `SWIMPAY\_STAGING\_WEBHOOK\_SECRET` in Android, browser JavaScript, Git, logs or screenshots.

2. Is this enough to integrate SwimPay SDK?

Yes, for the SwimPay side, the Developer Integration values are sufficient to integrate:

? server-side order creation;

? hosted checkout opening;

? webhook verification;

? final fulfillment after `payment.confirmed`;

? buyer-facing web button that calls the merchant backend;

? Android app checkout button through `SwimPayButton`;

? Android app checkout launch through `checkout\_url`.

The external developer still needs normal app-owned values that SwimPay cannot generate:

? the merchant app backend URL;

? the merchant order id;

? the merchant return URL or Android return scheme;

? the merchant fulfillment logic;

? a public HTTPS webhook endpoint.

So the rule is:

SwimPay generates the credentials.

The merchant developer owns the app backend, order model, return URL and fulfillment behavior.

3. Recommended architecture

Web or backend app

Browser / frontend

-> merchant backend

- > @swimpay/node creates order
- > backend returns checkout_url
- > browser redirects to checkout_url
- > SwimPay hosted checkout
- > merchant manually confirms review in SwimPay
- > SwimPay sends signed webhook to merchant backend
- > backend verifies webhook
- > backend fulfills merchant order

The browser never receives a SwimPay secret.

External Android merchant app

Android app

- > merchant backend creates SwimPay order
- > backend returns checkout_url
- > Android opens checkout_url with SwimPayCheckout
- > checkout returns to Android return scheme
- > Android refreshes order status from merchant backend
- > merchant backend waits for verified SwimPay webhook
- > backend fulfills only after payment.confirmed

The Android app never receives a SwimPay secret.

The Android return does not confirm payment. It only means the checkout browser returned control to the app.

4. Backend installation

Install the Node SDK in the merchant backend:

```
npm install @swimpay/node
```

Recommended backend env:

```
SWIMPAY_API_BASE_URL=https://staging.swimpay.pro
SWIMPAY_SECRET_KEY=sk...
SWIMPAY_WEBHOOK_SECRET=whsec...
EXTERNAL_APP_BASE_URL=https://your-backend.example
```

You can map the wizard values directly:

```
SWIMPAY_API_BASE_URL=$SWIMPAY_STAGING_API_BASE_URL
SWIMPAY_SECRET_KEY=$SWIMPAY_STAGING_SECRET_KEY
SWIMPAY_WEBHOOK_SECRET=$SWIMPAY_STAGING_WEBHOOK_SECRET
```

5. Create a SwimPay order from your backend

Example:

```
import { SwimPay } from "@swimpay/node";

const swimpay = new SwimPay({
  secretKey: process.env.SWIMPAY_SECRET_KEY!,
  apiBaseUrl: process.env.SWIMPAY_API_BASE_URL
});

export async function createSwimPayCheckout(order: {
  id: string;
  amountMinor: number;
  currency: "RUB";
}) {
  const checkout = await swimpay.orders.create(
    {
      externalOrderId: order.id,
      amountMinor: order.amountMinor,
      currency: order.currency,
      returnUrl: `${process.env.EXTERNAL_APP_BASE_URL}/orders/${order.id}/return`,
      metadata: {
        source: "merchant_backend"
      }
    },
    {
      idempotencyKey: order.id
    }
  );

  return {
    orderId: checkout.orderId,
    paymentSessionId: checkout.paymentSessionId,
    checkoutUrl: checkout.checkoutUrl,
    status: checkout.status,
    expiresAt: checkout.expiresAt
  };
}
```

Rules:

- ? Use one idempotency key per merchant order.
- ? Use `amountMinor` in minor currency units. Example: `42500` means `425.00 RUB`.
- ? Do not send CVV, card expiry, card PIN, bank password or SMS code.
- ? Do not send raw notification text in metadata.

? Do not ask SwimPay to auto-confirm. V1 does not support that.

6. Web frontend redirect

Your frontend should call your backend first. The backend returns only `checkoutUrl`.

Buyer-facing web button

```
<style>
.swimpay-button {
  min-height: 56px;
  border: 0;
  border-radius: 18px;
  padding: 0 24px;
  color: white;
  background: linear-gradient(135deg, #0097A7, #00698B);
  font: 800 16px/1 system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  cursor: pointer;
  box-shadow: 0 14px 32px rgba(0, 151, 167, 0.26);
}
.swimpay-button:disabled {
  cursor: progress;
  opacity: 0.72;
}
</style>
<button id="swimpay-button" class="swimpay-button" type="button">Payer avec SwimPay</button>

const button = document.getElementById("swimpay-button") as HTMLButtonElement;

button.addEventListener("click", async () => {
  button.disabled = true;
  try {
    const orderId = getCurrentOrderId();
    const response = await fetch(`/api/orders/${orderId}/swimpay-checkout`, {
      method: "POST"
    });
    const checkout = await response.json() as { checkoutUrl: string };
    window.location.assign(checkout.checkoutUrl);
  } finally {
    button.disabled = false;
  }
});
```

The button never receives SwimPay secrets. It only asks the merchant backend to create the checkout.

```

async function startPayment(orderId: string) {
  const response = await fetch(`/api/orders/${orderId}/swimpay-checkout`, {
    method: "POST"
  });

  const checkout = await response.json();
  window.location.assign(checkout.checkoutUrl);
}

```

Do not create the SwimPay order from browser JavaScript.

7. Android app integration

The Android helper is currently a Kotlin source helper in:

```
packages/swimpay-android/src/main/kotlin/com/swimpay/sdk/SwimPayCheckout.kt
```

It is intentionally small:

- ? shows a buyer-facing `SwimPayButton`;
- ? opens `checkout\_url`;
- ? parses return intent/URI;
- ? tells the app to refresh backend status;
- ? does not contain secrets;
- ? does not handle webhooks;
- ? does not confirm payments.

Android manifest return scheme

Example:

```

<activity
  android:name=".CheckoutActivity"
  android:exported="true"
  android:launchMode="singleTop">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="merchantapp" android:host="swimpay-return" />
  </intent-filter>
</activity>

```

Open checkout from Android

```
class CheckoutActivity : AppCompatActivity() {
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    val swimPayButton = SwimPayButton.create(this) { view ->
        val button = view as Button
        SwimPayButton.bind(button, SwimPayButtonState.Loading)

        lifecycleScope.launch {
            val checkoutUrl = merchantBackend.createSwimPayCheckout(orderId)
            val result = SwimPayCheckout.open(
                activity = this@CheckoutActivity,
                checkoutUrl = checkoutUrl,
                options = SwimPayCheckoutOptions(
                    returnScheme = "merchantapp",
                    allowedHosts = setOf("staging.swimpay.pro")
                )
            )

            SwimPayButton.bind(button, SwimPayButtonState.Ready)
            if (result.error != null) {
                showCheckoutError(result.safeMessage)
            }
        }
    }

    setContentView(swimPayButton)
}

fun payWithSwimPay(orderId: String) {
    lifecycleScope.launch {
        val checkoutUrl = merchantBackend.createSwimPayCheckout(orderId)

        val result = SwimPayCheckout.open(
            activity = this@CheckoutActivity,
            checkoutUrl = checkoutUrl,
            options = SwimPayCheckoutOptions(
                returnScheme = "merchantapp",
                allowedHosts = setOf("staging.swimpay.pro")
            )
        )

        if (result.error != null) {
            showCheckoutError(result.safeMessage)
        }
    }
}

```

```

    }
}

override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)

    val result = SwimPayCheckout.parseReturnIntent(
        intent,
        SwimPayCheckoutOptions(returnScheme = "merchantapp")
    )

    if (result?.shouldRefreshBackend == true) {
        refreshOrderStatusFromBackend()
    }
}
}

```

Android rule:

Return from checkout != paid.

Only your backend can mark the merchant order fulfilled, after a verified payment.confirmed webhook.

Android returnScheme wins over web_return_url/return_url.

web_return_url/return_url is only a browser fallback.

8. Webhook endpoint

Your backend must expose the configured `SWIMPAY\_WEBHOOK\_URL`.

Route accuracy is mandatory:

? `SWIMPAY\_WEBHOOK\_URL` must match the exact public backend route.

? If your backend uses a global prefix/version (`/api`, `/v1`, `/api/v1`), include it in the URL.

? A partial route (wrong prefix/path) usually returns HTTP `404` and keeps webhook deliveries in retry.

Example with a prefixed backend route:

```
SWIMPAY_WEBHOOK_URL=https://api.your-site.com/api/v1/payments/swimpay/webhook
```

Example with Express:

```

import express from "express";
import { SwimPay } from "@swimpay/node";

const app = express();

const swimpay = new SwimPay({

```



```

    secretKey: process.env.SWIMPAY_SECRET_KEY!,
    apiBaseUrl: process.env.SWIMPAY_API_BASE_URL
  });

app.post(
  "/webhooks/swimpay",
  express.raw({ type: "application/json" }),
  async (req, res) => {
    const event = swimpay.webhooks.verify(
      req.body,
      req.headers,
      process.env.SWIMPAY_WEBHOOK_SECRET!
    );

    await saveWebhookEventIfNew(event.id);

    switch (event.type) {
      case "payment.confirmed":
        await fulfillOrder(event.data.externalOrderId ?? event.data.orderId);
        break;
      case "payment.rejected":
        await markOrderRejected(event.data.externalOrderId ?? event.data.orderId);
        break;
      case "payment.expired":
        await markOrderExpired(event.data.externalOrderId ?? event.data.orderId);
        break;
    }

    res.sendStatus(200);
  }
);

```

Critical webhook rules:

- ? Verify the signature on the raw request body.
- ? Use `SwimPay-Event-Id` as the idempotency key.
- ? Return `2xx` only after your backend safely accepts the event.
- ? Fulfill only after verified `payment.confirmed`.
- ? Never fulfill from checkout return, `signal\_detected`, `needs\_review`, matching score or Android local state.

9. Public webhook events

V1 public fulfillment events:

```
payment.confirmed
payment.rejected
payment.expired
```

Public event data contains the operational confirmation boundary:

```
{
  "type": "payment.confirmed",
  "data": {
    "confirmation_type": "notification_signal",
    "official_bank_confirmation": false,
    "decision": "manual_confirmed"
  }
}
```

Internal SwimPay states such as signal detected, matching started or review created are not public fulfillment events.

10. Merchant prerequisites in SwimPay

Before a developer test can work end-to-end, the merchant must have:

1. A valid Android merchant session.
2. At least one active receiving method:
 - card;
 - or phone transfer wording used by users familiar with SBP-style phone transfer habits.
3. A bank associated with that receiving method.
4. Receiver configured and bank targets selected.
5. Developer Integration configured:
 - API key created and copied show-once;
 - webhook secret created and copied show-once;
 - webhook URL saved;
 - test webhook passing.

If no active receiving method exists, checkout should fail or show merchant configuration incomplete.

11. Staging test checklist

Use this order:

1. Open Developer Integration in SwimPay merchant app.
2. Create or rotate API key.
3. Create or rotate webhook secret.
4. Copy the full export block immediately.
5. Put the values only in the merchant backend environment.

6. Save the public webhook URL in SwimPay.
7. Run SwimPay test webhook.
8. Create an order through the merchant backend.
9. Open `checkout\_url` from web or Android.
10. Complete buyer checkout flow.
11. Confirm manually in SwimPay merchant review.
12. Verify the merchant backend receives `payment.confirmed`.
13. Fulfill the merchant order only after signature verification.

12. Common mistakes

Using masked secrets

Wrong:

```
SWIMPAY_SECRET_KEY=sk_****1234
```

Masked values are display-only. Rotate/create the secret and copy the show-once value.

Putting secrets in Android

Wrong:

```
val swimpaySecret = "sk_..."
```

The Android app must call the merchant backend. The backend owns SwimPay secrets.

Fulfillment after Android return

Wrong:

```
if (result.status == SwimPayCheckoutStatus.Returned) {  
    markOrderPaid()  
}
```

Correct:

```
if (result.shouldRefreshBackend) {  
    refreshOrderStatusFromBackend()  
}
```

Fulfillment without webhook verification

Wrong:

```
const event = JSON.parse(req.body.toString());  
fulfillOrder(event.data.order_id);
```

Correct:

```
const event = swimpay.webhooks.verify(  
  req.body,  
  req.headers,  
  process.env.SWIMPAY_WEBHOOK_SECRET!  
);
```

13. Final integration contract

The developer can integrate SwimPay when they have:

- ? `SWIMPAY\_STAGING\_API\_BASE\_URL`;
- ? full `SWIMPAY\_STAGING\_SECRET\_KEY`;
- ? full `SWIMPAY\_STAGING\_WEBHOOK\_SECRET`;
- ? configured `SWIMPAY\_WEBHOOK\_URL`;
- ? their own `EXTERNAL\_APP\_BASE\_URL`;
- ? a backend endpoint that creates orders;
- ? a backend endpoint that verifies webhooks;
- ? a web or Android client that opens only `checkout\_url`.

That is enough for SDK integration.

It is not enough to bypass the V1 product truth:

- ? no automatic confirmation;
- ? no official bank confirmation;
- ? no Android fulfillment;
- ? no browser/mobile secret;
- ? no fulfillment before verified final webhook.